



STATIC DESIGN PART 1

Software Design

Fontys University of Applied Science

14-11-2021

Stijlaart, Casey C.

466488

Table of Contents

1. Part 1	2
1.1. Use case definition	2
Primary function	2
Secondary function	2
Reasoning	2
Actors	2
Use Case(s)	2
1.2. Working towards a class diagram	3
Object discovery	3
Association discovery	3
First class diagram	4
Completion first class diagram	5
Required operations per class	5
Verify interactions between classes	6

Figures

Figure 1. Use case diagram	2
Figure 2. Sequence diagram request elevator	3
Figure 3. Object diagram elevator system	3
Figure 4. First class diagram	4
Figure 5. Completion class diagram	5
Figure 6. Extended class diagram with operations	6
Figure 7. Sequence diagram scenario 1	6
Figure 8. Sequence diagram scenario 2	7
Figure 9. Sequence diagram scenario 3	7

Tables

Table 1. UC_001	2
Table 2. UC_002	2
Table 3. UC_003	2
Table 4. Classes and their responsibilities	4
Table 5. Operations per class	5

1. Part 1

1.1. Use case definition

Primary function

The primary function of the system is to transport the user from one floor to another.

Secondary function

- Operate in a safe manner.
- The user gets the most suitable elevator, based on their input.
- The user shall be able to get an indication of the status of the elevator.
- The user shall be able to get an indication when their call for an elevator has been successful.

Reasoning

The system is being built to improve the efficiency of the current elevator system. The current elevator system allows the user to get on the elevator at any given moment that it passes their floor, regardless of their wished for direction and the elevators direction.

Actors

Regarding to this system, there is only one primary actor, which is the user itself. There is a secondary actor as well, which is the display to indicate on which floor the elevator is currently on. The secondary actor is only relevant when the secondary functions are taken into account. Lastly, the request buttons and floor buttons can be seen as external actors, since the user has to physically interact with them.

Use Case(s)

When taking only the primary function in to account, there are a few use cases which determine how the system will be functioning. See Table 1. UC_001, Table 2. UC_002, Table 3. UC_003, and **Error! Reference source not found.**

Use Case ID: UC_001	Use Case: Use the elevator.
Description	The user wants to ride the elevator when they have selected their desired floor.
Actor:	An elevator user.

Table 1. UC_001

Use Case ID: UC_002	Use Case: Select a floor.
Description	The user can pick their desired floor, based on the displayed floor options.
Actor:	An elevator user.

Table 2. UC_002

Use Case ID: UC_003	Use Case: Request an elevator.
Description	The user wants to ride the elevator, they will have to request an elevator to come to their floor.
Actor:	An elevator user.

Table 3. UC_003

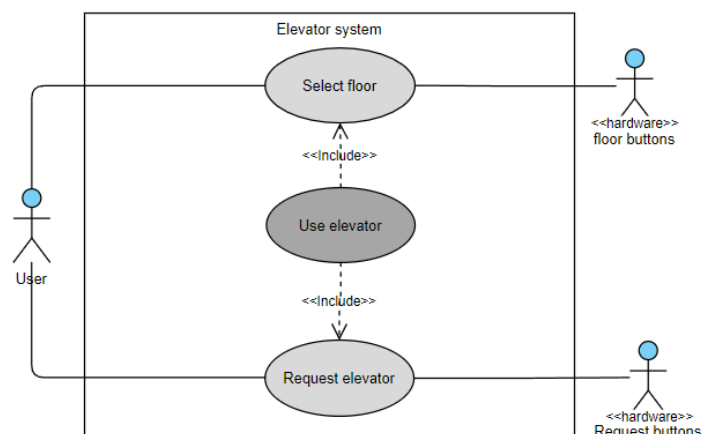


Figure 1. Use case diagram

1.2. Working towards a class diagram

Object discovery

There are multiple objects when it comes to the elevator. These objects are as following, the request button up, request button down, floor button, elevator door, floor door, elevator, Cable tension sensor and the general controller.

The reason for also including a general controller, is since there are multiple elevators within the building, and they need to be managed and keep track of to find the best suitable elevator for the user.

Regarding to the two doors, since both work separately and can have their own errors while running, I decided to make them each their own object.

Association discovery

To get a better understanding of how the objects are associated with each other, a sequence diagram, for when an elevator is being requested, has been provided. See Figure 2. Sequence diagram request elevator. The object diagram is shown in Figure 3. Object diagram elevator system.

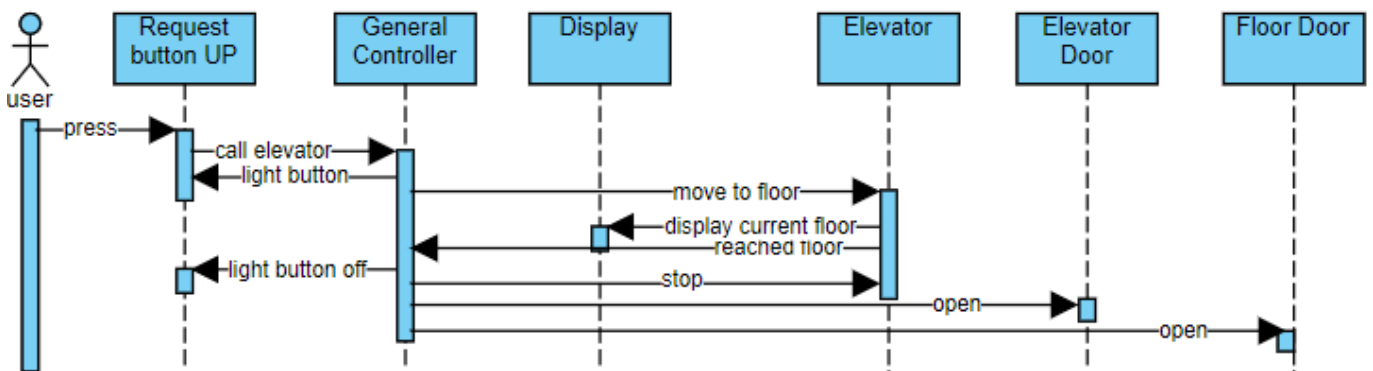


Figure 2. Sequence diagram request elevator

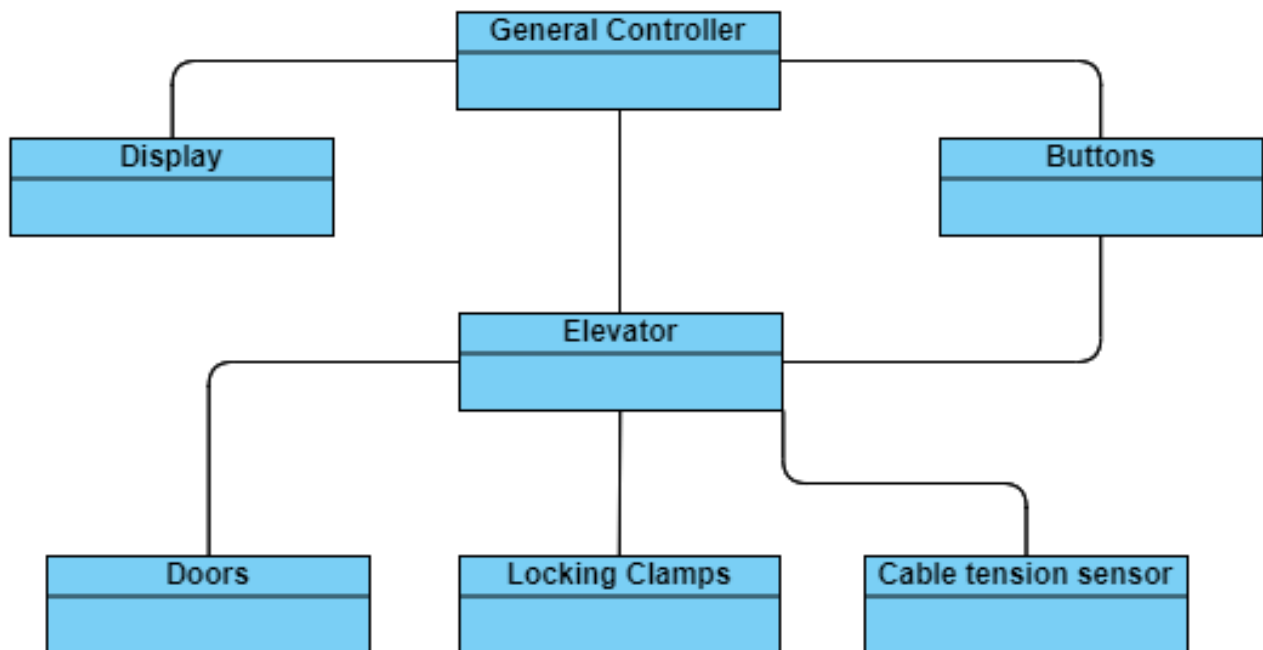


Figure 3. Object diagram elevator system

First class diagram

Based on the previous defined objects, see Figure 3. Object diagram elevator system, the first decisions for the class diagram can be made. See Figure 4. First class diagram. In this diagram I've made the decision to not include the inheritance for the buttons and the doors yet and have made it more abstract by combining them into interface classes, buttons and doors. In the next heading the inheritance will be considered.

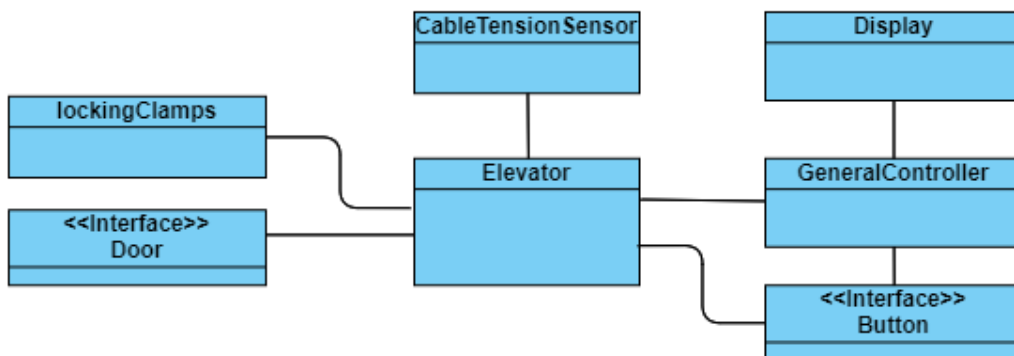


Figure 4. First class diagram

Class	Responsibilities
Class Display	- Displays the current floor information, based on a status update from the elevator.
Class Floor Door	- Opens and closes the door based on the floor number and elevator id.
Class Elevator Door	- Opens and closes the elevator door based on the elevator id.
Class General Controller	- Receives the information from the buttons and makes the decision which elevator to send based on the request and the best suitable elevator, based on their status. - Each floor has their own general controller. - Informs the maintainer when the cable tension has exceeded its limit.
Class Elevator	- Keeps track of the current direction and current floor of the elevator. - Moves to a requested floor based on the requested floor number and if their elevator ID has been called. - Stops moving when the requested floor has been reached. - Sends out their status (current floor, direction) to the general controller when requested. - Sends out their status (current floor, direction) to the display when their status has been updated.
Class Tension Sensor	- Keeps track of the value situation on the tension sensor per elevator id. - Returns its status to the locking clamps and general Controller when it has reached a critical value.
Class Locking Clamps	- The clamps will be locked when it receives a message from the cable tension sensor.
Button Floor Button	- Illuminates when the button has been pressed. - Stops illuminating when the elevator has reached the requested floor number. - Sends out an request to the move the elevator to floor
Button Request Button UP	- Illuminates when the button has been pressed. - Stops illuminating when the elevator has reached the floor id from the button. - Sends out an elevator request to their floor number and with an upwards direction.
Button Request Button DOWN	- Illuminates when the button has been pressed. - Stops illuminating when the elevator has reached the floor id from the button. - Sends out an elevator request to their floor number and with a downwards direction.

Table 4. Classes and their responsibilities

Completion first class diagram

Based on the first initial class diagram, Figure 4. First class diagram, and the described responsibilities per class, Table 4. Classes and their responsibilities, the first class diagram can be extended in such a way that the attributes per class are defined, see

Regarding to the buttons, they all serve the same main function, when it's pressed send the information, these can use inheritance from a button interface. This will make it much easier to program, since you will not have repeated the same function over and over. The same goes for the doors, both have the same main function; open and close. Each class has their own responsibilities, see Table 4. Classes and their responsibilities.

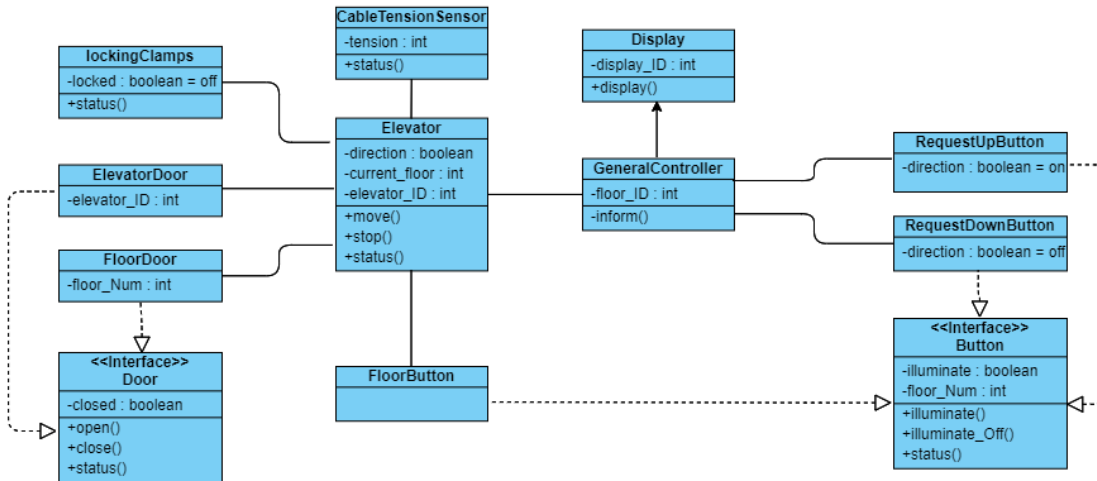


Figure 5. Completion class diagram

Required operations per class

Each class has their own set of operations, which they carry out. To get a better and clearer overview on these operations, I've decided on first making a table for all the operations per class and their description. For most of the described class in the given table, Table 5. Operations per class, I've added a status operation. This status operation serves as purpose to allow the connected classes to retrieve the data of the specific class. In Figure 6 there is new version of the class diagram given which includes the operations.

Class	Operation	Description
GeneralController	inform()	Sends out an inform message to the maintainer when a status message from the cable tension sensor has been received.
Elevator	move()	Moves the elevator from their current floor in the direction of the target floor.
Elevator	stop()	Stops the elevator from moving.
Elevator	status()	Gets the current status.
Door	close()	Closes the door.
Door	open()	Opens the door.
Door	status()	Gets the current status.
Button	illuminate()	Turns on the backlight of the button.
Button	illuminate_Off()	Turns off the backlight of the button.
Button	status()	Gets the current status.
CableTensionSensor	status()	Gets the current status.
Display	display()	Displays the current status of the elevator.

Table 5. Operations per class

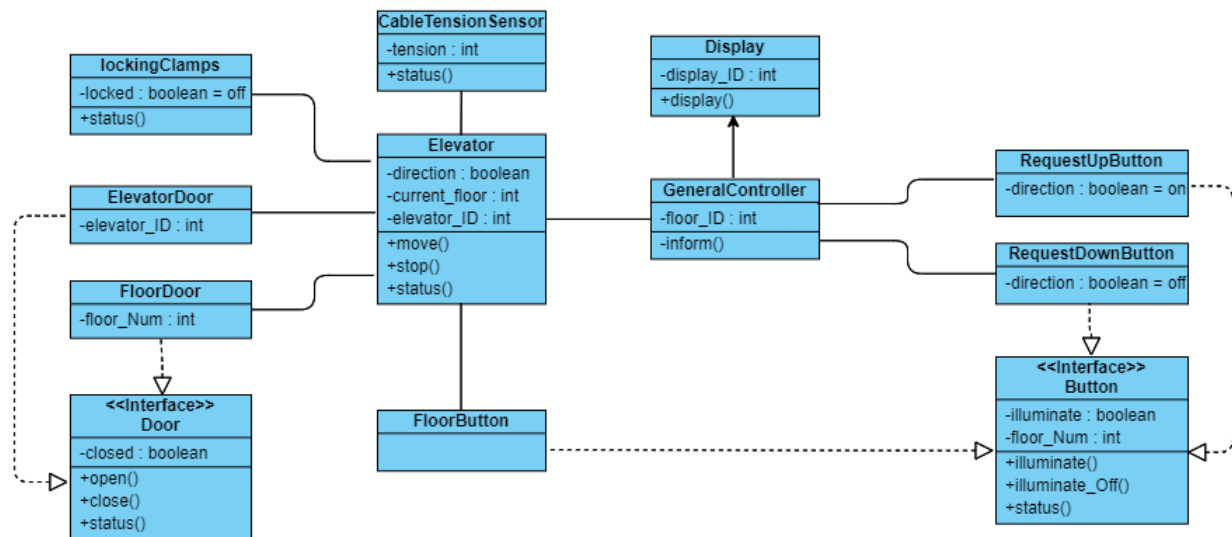


Figure 6. Extended class diagram with operations

Verify interactions between classes

To verify the created class diagram, there can be sequence diagrams made for specific scenarios. There are three different scenarios' given.

Situation 1 : The user ride the lift to the requested floor.

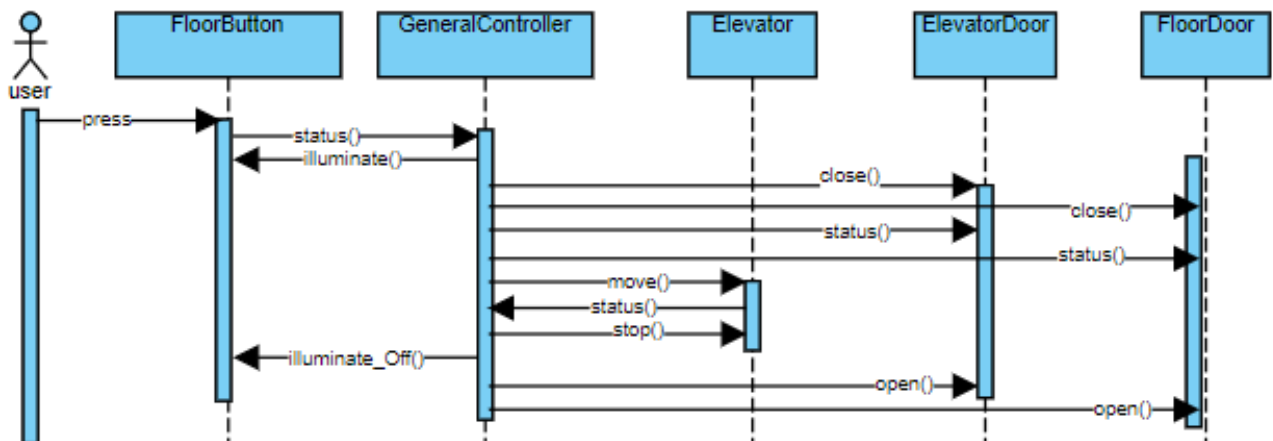


Figure 7. Sequence diagram scenario 1

Situation 2 : The user presses the request down button, and watches the display updating the current elevator status until the elevator arrives.

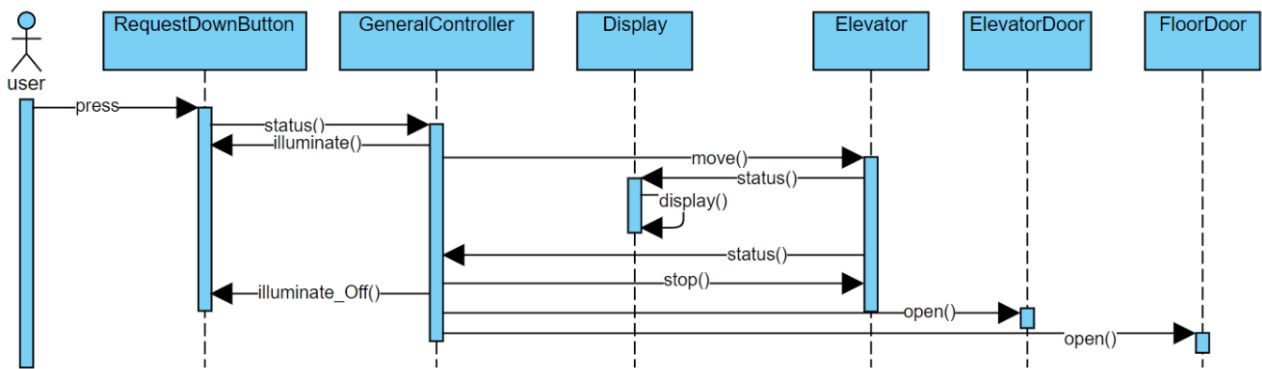


Figure 8. Sequence diagram scenario 2

Situation 3 : There is too much tension on the cable tension sensor, therefore the locking clamps are locked and the elevator is brought to a halt.

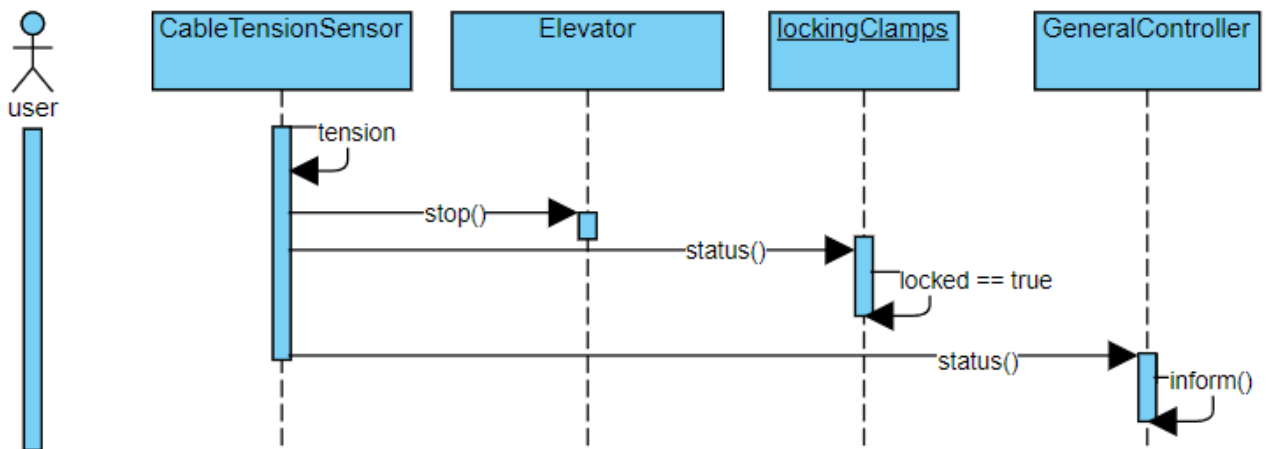


Figure 9. Sequence diagram scenario 3